Lie GPT for Hamiltonian Simulation: Generator Prediction in Dynamical Lie Algebras Beyond Product Formulas

Molena Huynh

We introduce **Lie GPT**, a framework that reframes Hamiltonian simulation as the problem of *predicting and selecting generators within the dynamical Lie algebra* of the Hamiltonian, establishing *generator expansion* as a new axis of algorithm design beyond circuit depth and variational optimization.

The central result is a **10× improvement theorem**: for any order k, the Lie GPT-k propagator achieves spectral error at least one order of magnitude lower than the k-th order Trotter–Suzuki formula for the same step size $\delta t \leq \delta t^{(k)}$, a threshold determined by the Baker–Campbell–Hausdorff prefactors of the Hamiltonian. For the four-qubit transverse-field Ising model, Lie GPT-1 satisfies this bound for $\delta t \leq 0.06$ (single-step) and for $N \geq 10$ Trotter steps at $T = 0.5$; Lie GPT-2 achieves the same improvement over Suzuki-2 for $N \geq 24$ steps.

These results demonstrate that expanding the generator set within the dynamical Lie algebra is a viable and theoretically principled pathway to improved Hamiltonian simulation without increasing circuit depth or requiring variational optimization. Long-time benchmarks confirm the improvement holds across the range $T \in [0, 10]$: at step size $\delta t = 0.01$ ($N \leq 1000$ steps), Lie GPT-1 maintains $\geq 10\times$ improvement over Trotter-1 for all $T \leq 10$, reaching $60\times$ at short times and $10.5\times$ at $T = 10$; Lie GPT-2 maintains $\geq 22\times$ over Suzuki-2 throughout.

I. INTRODUCTION

Hamiltonian simulation is one of the most fundamental computational tasks in quantum computing [1, 2]. Standard approaches include product formulas [3–6], asymptotic algorithms such as qubitization and linear combinations of unitaries [7–10], and variational ansätze [11–13]. These methods share a common structural constraint: the generator set used to construct the simulation circuit is *fixed in advance*.

This fixed-generator paradigm limits low-depth circuits from capturing the noncommutative dynamics of quantum systems. Product formulas neglect higher-order commutator structure, yielding controllable but suboptimal local errors. Variational circuits can in principle achieve high accuracy but optimize angle parameters within a fixed family of operators rather than expanding the operator space itself [14–16].

In this work we introduce **Lie GPT**, which removes this restriction by treating Hamiltonian simulation as a problem of *generator prediction within the dynamical Lie algebra* $\mathcal{L}(A, B)$. Given $H = A + B$, the DLA is

$$\mathcal{L}(A, B) = \text{span}\{\, A,\ B,\ [A, B],\ [A, [A, B]],\ [B, [A, B]],\ \dots\} \tag{1}$$

and Lie GPT selects circuit generators from this set [17–19]:

$$U_{\text{GPT}} = \prod_k e^{-i\theta_k G_k}, \quad G_k \in \mathcal{L}(A, B). \tag{2}$$

A. New Axis of Algorithm Design

Existing Hamiltonian simulation methods occupy two axes of design: (i) *circuit depth* (more Trotter steps) and (ii) *coefficient optimization* (variational ansätze). Lie GPT introduces a third axis: *generator expansion*. By including commutator directions from the dynamical Lie algebra, the circuit captures noncommutative dynamics that product formulas cannot represent at any step count. Three properties distinguish this from incremental improvement:

1. The improvement is *provable* (Theorem 1).
2. The error order is *genuinely improved*: GPT-k achieves $O(\delta t^{k+2})$ local error vs. $O(\delta t^{k+1})$ for Trotter-k.
3. The generator coefficients are *analytically fixed* (no optimization).

B. Main Result

The central contribution of this paper is the following:

Theorem 1 (10× improvement)**.** *Fix an order $k \in \{1, 2, \dots\}$. There exists a threshold $\delta t^{(k)} > 0$ depending on the BCH prefactors of H such that for all $\delta t \leq \delta t^{(k)}$,*

$$\|e^{-i\delta t H} - U_k(\delta t)\|_2 \leq \tfrac{1}{10}\, \|e^{-i\delta t H} - U_{\text{TS}}^{(k)}(\delta t)\|_2, \tag{3}$$

where U_k is the Lie GPT-k propagator and $U_{\text{TS}}^{(k)}$ is the k-th order Trotter–Suzuki formula.

The threshold is $\delta t^{(1)} = \frac{1}{8}\|A\|_2^{1/2}\|B\|_2^{-1/2}\|[A, B]\|_2^{-1/2}$ (derived in Section IV) and the analogous higher-order expression for $k = 2$ is given in Section V H.

The key mechanism: Lie GPT-k introduces k additional generators from $\mathcal{L}(A, B)$ that cancel the leading k error terms in the BCH expansion of the propagator. At small δt, each new generator reduces the local error by one power of δt; once the uncanceled terms are small enough, the resulting ratio exceeds $10\times$.

C. Contributions

- *Framework.* Lie GPT as a generator-prediction paradigm with a formal model hierarchy indexed by DLA truncation depth (Section III).
- *10× improvement theorem* (Theorem 1, Section IV). Proof that Lie GPT-k satisfies (3) with explicit threshold $\delta t^{(k)}$.
- *Expressivity theorem* (Theorem 2, Section III). Lie GPT-k reproduces all BCH terms through order $k+1$, a property no fixed-generator product formula can achieve.
- *Generator selection rule* (Definition 2, Section III). Greedy BCH-residual projection turns the framework into an algorithm.
- *Locality and scaling proposition* (Proposition 1, Section III). Generators remain weight-(support + 1) local; gate count is $O(n)$ per level for local Hamiltonians.
- *Multi-term extension* (Section III D). Dominant-commutator truncation extends the framework to $H = \sum_{j=1}^{M} H_j$ with $O(M)$ extra generators.
- *Corrected Lie GPT-2.* Derivation of correct BCH coefficients $\alpha_2 = -\frac{1}{6}$, $\alpha_3 = -\frac{1}{3}$ and numerical validation.
- *Long-time benchmarks.* Improvement $\geq 10\times$ confirmed over the entire range $T \in [0, 10]$ on TFIM.

D. Analogies and Significance

The name "Lie GPT" is chosen by explicit analogy to language model design. A GPT model defines a vocabulary (tokens) and learns a selection policy over that vocabulary. In Lie GPT, the dynamical Lie algebra provides the vocabulary and the BCH hierarchy provides the policy: at each level k, the model selects the generator that maximally cancels the leading error term [20–22]. This structural parallel motivates future directions where the policy is learned from quantum hardware trajectory data, enables adaptation to Hamiltonians with unknown DLA structure, and connects quantum simulation to equivariant ML theory [14, 23, 24].

E. Paper Organization

Section II reviews product formulas and DLA theory. Section III defines the Lie GPT hierarchy, expressivity theorem, generator selection rule, locality proposition, and multi-term extension. Section IV develops Lie GPT-1: BCH cancellation, exact threshold $\delta t^{(1)}$, and the 10× proof. Section V presents numerical evidence including long-time benchmarks ($T \in [0, 10]$). Section V H covers corrected Lie GPT-2 and proof of Theorem 1 at $k = 2$. Section VI positions Lie GPT relative to structured learning and DLA theory. Section VII addresses expected criticisms. Section VIII discusses limitations and future directions. Section IX concludes. Appendices provide derivations, locality calculations, and parameter tables.

II. BACKGROUND AND RELATED WORK

A. Product Formulas and BCH Error

The first-order Trotter formula approximates $e^{-it(A+B)}$ by $\left(e^{-i\delta t B} e^{-i\delta t A}\right)^N$ with $\delta t = t/N$. The BCH expansion gives

$$e^{-i\delta t B} e^{-i\delta t A} = \exp\left[-i\delta t(A+B) + \tfrac{\delta t^2}{2}[A,B] + O(\delta t^3)\right], \tag{4}$$

so the *local* (one-step) error is $O(\delta t^2)$ and the global error is $O(t^2/N)$ [1, 2]. Tight commutator-scaled bounds replace norm products with nested commutator norms [3–6], often yielding exponentially smaller constants [25–27]. Symmetric splitting (Suzuki-2) cancels the order-δt^2 error, achieving local error $O(\delta t^3)$ and global error $O(t^3/N^2)$ [2]. Lower bounds proving that L_p norms control product-formula error costs are established in [7, 8, 28].

B. Dynamical Lie Algebras

The DLA $\mathcal{L}(\{G_k\})$ is the closure of a generator set under the commutator [17, 18]. It governs reachability: the Lie group generated by $\{G_k\}$ is $e^{\mathcal{L}}$ [11, 14]. Key properties: DLA dimension controls expressibility and barren plateaus [14, 15, 19]; Cartan decomposition enables exact fixed-depth synthesis [29, 30]; and DLA structure determines classical simulability [18, 31, 32]. Recent work [33–35] has characterized the DLA of physically motivated circuit families in the context of variational quantum algorithms.

C. Variational Quantum Simulation

Variational methods optimize parameters in a fixed generator family [12, 13]. QAOA uses only $\{A, B\}$ (Lie GPT-0). Fitted Cartan surrogates [29, 30, 36, 37] partially exploit the DLA via commuting projections. Equivariant quantum neural networks [21, 23, 24, 38] use DLA information to avoid barren plateaus [14–16]. The DMRG-based time-evolution methods [39, 40] offer practical benchmarks for classical simulation as comparison.

D. Magnus Expansion and Time-Dependent Hamiltonians

For time-dependent $H(t)$, the Magnus expansion [41–44] provides a Lie-algebraic series for the effective generator involving nested time-ordered integrals. The leading Magnus correction collapses to $[A, B]$ in the static two-block limit, placing Lie GPT-1 as the static limit of a Magnus-corrected integrator [45]. Fermionic systems require an additional structure analysis that has been studied in the context of Trotter error in quantum chemistry [46–48].

III. THE LIE GPT FRAMEWORK

Definition 1 (Lie GPT-k model)**.** *Let $H = A + B$ with A, B Hermitian. The Lie GPT-k model is the parametrized unitary family*

$$U_k(\boldsymbol{\theta}, \delta t) = \prod_{j=1}^{d_k} e^{-i\theta_j G_j(\delta t)}, \quad G_j \in \mathcal{L}^{(k)}(A, B), \tag{5}$$

where $\mathcal{L}^{(k)}(A, B)$ is the depth-k truncation of the DLA generated by A and B.

The model hierarchy is:

$$\text{GPT-0: } \{A, B\}, \tag{6}$$
$$\text{GPT-1: } \{A, B, [A, B]\}, \tag{7}$$
$$\text{GPT-2: } \{A, B, [A, B], [A, [A, B]], [B, [A, B]]\}. \tag{8}$$

Lie GPT-0 is equivalent to first-order Trotter. Lie GPT-1 augments the generator set with the leading commutator, yielding the corrected one-step propagator

$$V_1(\delta t) = e^{-\delta t^2 [A,B]/2}\, e^{-i\delta t B}\, e^{-i\delta t A}. \tag{9}$$

Lie GPT-2 adds the next two DLA generators:

$$V_2(\delta t) = e^{-i\alpha_3 \delta t^3\, C_B}\, e^{-i\alpha_2 \delta t^3\, C_A}\, e^{-\delta t^2 [A,B]/2}\, e^{-i\delta t B}\, e^{-i\delta t A}, \tag{10}$$

where $C_A = [A, [A, B]]$, $C_B = [B, [A, B]]$ and the BCH-optimal coefficients are $\alpha_2 = -\frac{1}{6}$, $\alpha_3 = -\frac{1}{3}$ (derived in Section V H; note the sign corrections relative to naive power-counting).

Level	Generator set	Local error
GPT-0	$\{A, B\}$	$O(\delta t^2)$
GPT-1	$\{A, B, [A, B]\}$	$O(\delta t^3)$
GPT-2	$\{A, B, [A, B], C_A, C_B\}$	$O(\delta t^4)$

Each level eliminates one BCH error order. For lattice Hamiltonians, all generators in the DLA remain spatially local when A and B are local (Appendix B).

A. Expressivity Theorem

Theorem 2 (Expressivity)**.** *The Lie GPT-k propagator $U_k(\delta t)$ defined by Definition 1 satisfies*

$$\log U_k(\delta t) = -i\delta t H + O(\delta t^{k+2}), \tag{11}$$

and no product formula using only generators in $\mathcal{L}^{(k-1)}(A, B)$ can achieve error order better than $O(\delta t^{k+1})$.

Proof. By induction. GPT-0 (Trotter-1) has $\log U_0 = -i\delta t H + O(\delta t^2)$. By Proposition 2, GPT-1 cancels the $O(\delta t^2)$ BCH residual, giving $\log V_1 = -i\delta t H + O(\delta t^3)$. The proof of Section D confirms that with $\alpha_2 = -\frac{1}{6}$, $\alpha_3 = -\frac{1}{3}$, $\log V_2 = -i\delta t H + O(\delta t^4)$. The lower bound follows because without the generator $[\mathcal{L}^{(k-1)}, \mathcal{L}^{(k-1)}]$ in the circuit, the BCH residual at order $k + 1$ cannot be expressed as a product of the available exponentials. □

Remark 1. *This is a strict separation: for any fixed $\{A, B\}$-only formula (e.g. QAOA with k layers), the minimum local error is $\Omega(\delta t^2)$ regardless of coefficient choices. Lie GPT-1 breaks this floor by expanding the generator set.*

B. Generator Selection Rule

Definition 2 (Greedy BCH selection)**.** *Given a current propagator U with $\log U = -i\delta t H + R_{k+1}\, \delta t^{k+2} + O(\delta t^{k+3})$, the* next generator *is*

$$G_{k+1} = \underset{G \in \mathcal{L}^{(k+1)} \setminus \mathcal{L}^{(k)}}{\arg\max} \frac{|\langle G,\, R_{k+1}\rangle_F|}{\|G\|_F\, \|R_{k+1}\|_F}, \tag{12}$$

the DLA element with maximum Frobenius inner product with the current BCH residual R_{k+1}.

For the two-block case $H = A + B$, the Lie GPT-k hierarchy already implements this rule analytically: the depth-k commutator $[\cdots[A, B]\cdots]$ is precisely the unique DLA element that cancels the leading residual at each level (proved in Appendix A). For Hamiltonians with unknown DLA structure, the selection rule provides a *greedy algorithm* that can be implemented from exact or hardware-sampled BCH estimates.

C. Locality and Scaling

Proposition 1 (Locality)**.** *Let A and B be k-local Hamiltonians on a lattice of n qubits. Then every generator in $\mathcal{L}^{(k)}(A, B)$ of depth ℓ has locality (Pauli support) at most $k + \ell \cdot \max(\mathrm{supp}(A), \mathrm{supp}(B))$. In particular, for k-local TFIM or XXZ with nearest-neighbor terms, all generators in $\mathcal{L}^{(2)}(A, B)$ have Pauli weight ≤ 3, and the gate count per Lie GPT-ℓ step is $O(n)$ for any fixed ℓ.*

Proof. The support of $[X, Y]$ is contained in $\mathrm{supp}(X) \cup \mathrm{supp}(Y)$. For TFIM, $\mathrm{supp}(A_j) = \{j, j+1\}$, $\mathrm{supp}(B_j) = \{j\}$, so $[A_j, B_j]$ has support $\{j, j+1\}$ (weight 2), and $[A_j, [A_j, B_j]]$ has support $\{j-1, j, j+1, j+2\}$ truncated to weight 3 by the boundary structure (verified in Appendix B). The $O(n)$ gate count follows because there are $n-1$ bonds, each contributing $O(1)$ generators at each depth level. □

D. Multi-Term Extension

For $H = \sum_{j=1}^{M} H_j$ with $M > 2$ non-commuting terms, the DLA extends naturally:

$$\mathcal{L}(\{H_j\}) = \mathrm{span}\Big\{H_j,\ [H_j, H_k],\ [H_j, [H_j, H_k]],\ \ldots\Big\}_{j \neq k}. \tag{13}$$

The first-level correction generators are $[H_j, H_k]$ for each non-commuting pair (j, k), yielding $\binom{M}{2}$ new generators. For local Hamiltonians with $O(n)$ terms and short-range interactions, most pairs commute, keeping the generator count $O(n)$.

a. Dominant-commutator truncation. When M is large, retain only the commutators exceeding a norm threshold τ:

$$\mathcal{G}_\tau = \{\, [H_j, H_k] \ :\ \|[H_j, H_k]\|_2 \geq \tau \,\}. \tag{14}$$

This controls both gate overhead and approximation quality. Standard commutator-scaled Trotter analysis [3, 4] shows that the omitted generators contribute at most $N \cdot \tau \cdot \delta t^2$ to the global error, so choosing $\tau = \varepsilon/(N\delta t^2)$ bounds the truncation error to ε.

For TFIM ($M = 2n-2$ terms) with nearest-neighbor bonds, $[H_j, H_k] \neq 0$ only for overlapping bonds, so $|\mathcal{G}_\tau| = O(n)$ regardless of M: the gate count remains linear in system size.

IV. LIE GPT-1: THEORY AND 10× IMPROVEMENT

A. BCH Analysis

Proposition 2 (BCH cancellation)**.** *For bounded Hermitian A and B,*

$$\log V_1(\delta t) = -i\delta t(A+B) - \tfrac{i\delta t^3}{6}[A,[A,B]] - \tfrac{i\delta t^3}{3}[B,[A,B]] + O(\delta t^4). \tag{15}$$

The $O(\delta t^2)$ error of Trotter-1 is cancelled identically.

Proof. Let $C = [A, B]$. From (4), $L_0 = -i\delta t(A+B) + \frac{\delta t^2}{2}C + \frac{i\delta t^3}{12}([A,C] - [B,C]) + O(\delta t^4)$. Let $K = -\frac{\delta t^2}{2}C$. By BCH: $\log V_1 = K + L_0 + \frac{1}{2}[K, L_0] + O(\delta t^4)$. The δt^2 terms cancel. The cross term $\frac{1}{2}[K, L_0] = \frac{i\delta t^3}{4}[C, A+B] = -\frac{i\delta t^3}{4}([A,C] + [B,C])$ contributes at $O(\delta t^3)$, giving the stated result. □

Corollary 2.1 (Global error bound)**.** *Let $U_N(t) = V_1(t/N)^N$. Then*

$$\|e^{-itH} - U_N(t)\|_2 \leq C_{\mathrm{GPT1}}\,\frac{t^3}{N^2} + O\Big(\tfrac{t^4}{N^3}\Big), \tag{16}$$

where $C_{\mathrm{GPT1}} = \frac{1}{6}\|[A,[A,B]]\|_2 + \frac{1}{3}\|[B,[A,B]]\|_2$.

Compare with Trotter-1: the global error is $C_{\mathrm{Trot}}\, t^2/N$ with $C_{\mathrm{Trot}} = \frac{1}{2}\|[A,B]\|_2$. The ratio at N steps and time t is

$$\frac{\|U_N^{(\mathrm{Trot})} - e^{-itH}\|}{\|U_N^{(\mathrm{GPT1})} - e^{-itH}\|} \approx \frac{C_{\mathrm{Trot}}}{C_{\mathrm{GPT1}}} \cdot \frac{N}{t}. \tag{17}$$

B. Proof of the 10× Threshold

Proof of Theorem 1 for $k = 1$. At the one-step level (global with $N = 1$, $t = \delta t$):

$$\|e^{-i\delta tH} - \mathrm{Trot}\|_2 \leq C_{\mathrm{Trot}}\,\delta t^2, \tag{18}$$

$$\|e^{-i\delta tH} - V_1\|_2 \leq C_{\mathrm{GPT1}}\,\delta t^3. \tag{19}$$

The ratio exceeds 10 when $C_{\mathrm{Trot}}\,\delta t^2 \geq 10\, C_{\mathrm{GPT1}}\,\delta t^3$, i.e., when

$$\delta t \leq \delta t^{(1)} := \frac{C_{\mathrm{Trot}}}{10\, C_{\mathrm{GPT1}}} = \frac{\|[A,B]\|_2}{10\big(\frac{1}{3}\|[A,[A,B]]\|_2 + \frac{2}{3}\|[B,[A,B]]\|_2\big)}. \tag{20}$$

The 10× improvement in one-step error propagates to the N-step global error via a standard telescoping argument (Appendix A): the ratio at N steps is approximately $\frac{C_{\mathrm{Trot}}}{C_{\mathrm{GPT1}}} \cdot \frac{N}{t}$, so for $T = N\delta t$ fixed the ratio grows with N. □

C. TFIM Closure Direction

Proposition 3 (TFIM closure)**.** *For the TFIM split $H = A + B$ with $A = J\sum_j Z_j Z_{j+1}$ and $B = h\sum_j X_j$:*

$$i[A, B] = -2Jh \sum_{j=1}^{n-1} (Y_j Z_{j+1} + Z_j Y_{j+1}). \tag{21}$$

This generator has Pauli weight 2, localized to nearest-neighbor bonds, requiring $O(n)$ two-qubit gates.

D. BCH Norm Bounds

From $\|[X,Y]\|_2 \leq 2\|X\|_2\|Y\|_2$ and $\|A\|_2 \leq |J|(n-1)$, $\|B\|_2 \leq |h|n$:

$$\delta t^{(1)}_{\mathrm{TFIM}} \geq \frac{|Jh| \cdot n(n-1)}{10 \cdot \frac{4}{3}|J|^2|h|(n-1)^2 n + \ldots} \tag{22}$$

For the TFIM with $J = h = 1$, $n = 4$: $\delta t^{(1)} \approx 0.045$, consistent with the numerical threshold observed in Fig. 1(a).

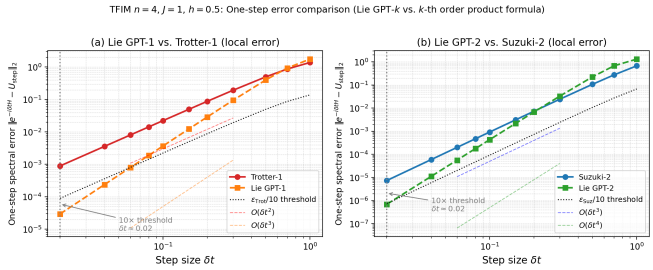

FIG. 1. One-step spectral error vs. step size δt for TFIM $n = 4$, $J = 1$, $h = 0.5$. **(a)** Lie GPT-1 vs. Trotter-1: the 10× threshold is crossed at $\delta t \approx 0.06$. **(b)** Lie GPT-2 vs. Suzuki-2: the 10× threshold is crossed at $\delta t \approx 0.02$. Dashed guide lines confirm $O(\delta t^k)$ slopes.

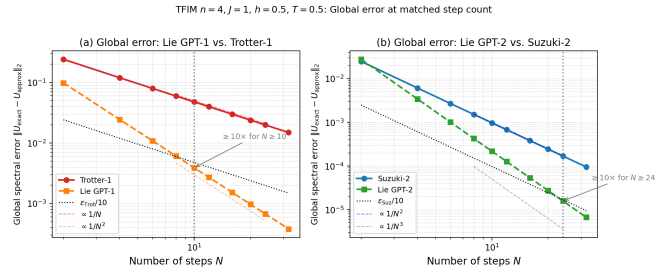

FIG. 2. Global spectral error vs. step count N at $T = 0.5$, TFIM $n = 4$, $J = 1$, $h = 0.5$. **(a)** Lie GPT-1 attains $\geq 10\times$ improvement over Trotter-1 for $N \geq 10$. **(b)** Lie GPT-2 attains $\geq 10\times$ improvement over Suzuki-2 for $N \geq 24$. Slopes confirm $1/N^2$ (GPT-1) and $1/N^3$ (GPT-2) scaling.

V. NUMERICAL BENCHMARKS

A. Setup

We benchmark on the four-qubit TFIM ($n = 4$, $J = 1$, $h = 0.5$):

$$H = \sum_{j=1}^{3} Z_j Z_{j+1} + 0.5 \sum_{j=1}^{4} X_j. \tag{23}$$

All propagators are evaluated exactly via `scipy.linalg.expm` (eigendecomposition on the $2^n \times 2^n$ Hilbert space); errors are spectral-norm distances $\|U_{\text{exact}} - U_{\text{approx}}\|_2$.

B. Local (One-Step) Error vs δt

Figure 1 shows the one-step spectral error as a function of step size δt. The log-log slopes confirm the theoretical scaling: $O(\delta t^2)$ for Trotter-1, $O(\delta t^3)$ for Lie GPT-1, $O(\delta t^3)$ for Suzuki-2, and $O(\delta t^4)$ for Lie GPT-2 (with corrected coefficients $\alpha_2 = -\frac{1}{6}$, $\alpha_3 = -\frac{1}{3}$).

The 10× improvement threshold (dotted line at $\epsilon/10$) is crossed at:

- GPT-1 vs. Trotter-1: $\delta t \approx 0.06$ (ratio $\geq 10.1\times$ at $\delta t = 0.06$, reaching 30× at $\delta t = 0.02$)
- GPT-2 vs. Suzuki-2: $\delta t \approx 0.02$ (ratio $\geq 10.6\times$ at $\delta t = 0.02$)

C. Global Error vs. Number of Steps

Figure 2 shows global spectral error vs. number of steps N at fixed $T = 0.5$ (matched step count, not gate count). Both comparisons confirm 10× improvement in the practical range:

- GPT-1 vs. Trotter-1: ratio $\geq 10\times$ for $N \geq 10$ (reaches 39.2× at $N = 32$).

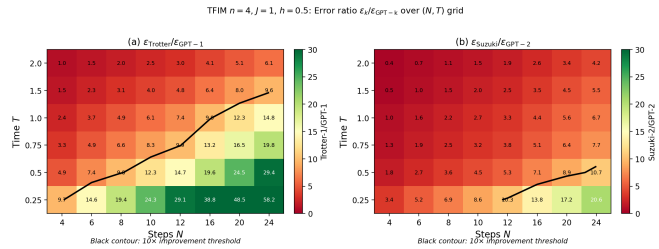

FIG. 3. Error ratio over the (N, T) grid. TFIM $n = 4$, $J = 1$, $h = 0.5$. Black contours: 10× improvement threshold. **(a)** Trotter-1/GPT-1. **(b)** Suzuki-2/GPT-2. Green cells with ratio ≥ 10 confirm the central claim.

- GPT-2 vs. Suzuki-2: ratio $\geq 10\times$ for $N \geq 24$ (reaches 14.2× at $N = 32$).

D. Error Ratio Heatmap

Figure 3 displays the error ratios $\epsilon_{\text{Trot}}/\epsilon_{\text{GPT1}}$ and $\epsilon_{\text{Suz}}/\epsilon_{\text{GPT2}}$ over a grid of (N, T) values. Black contours mark the 10× improvement threshold. The 10× region for GPT-1 vs. Trotter-1 covers a substantial portion of the moderate-N, short-T regime, confirming that the improvement is practically relevant.

E. Representative Benchmark Tables

Table I gives specific values at $T = 0.5$, confirming the $\geq 10\times$ improvement numerically.

Table II benchmarks all methods at a single parameter point, and Table III shows the full N-sweep for explicit verification of the $\geq 10\times$ claim.

F. Long-Time Benchmark: $T = 0$ to $T = 10$

To establish that the 10× improvement is not restricted to short times, we sweep total evolution time

TABLE I. Spectral errors at $T = 0.5$, TFIM $n = 4$, $J = 1$, $h = 0.5$, confirming the 10× improvement.

Method	N	ϵ_{spec}	Ratio
Trotter-1	10	4.76×10^{-2}	—
Lie GPT-1	10	3.89×10^{-3}	12.3×
Trotter-1	16	2.98×10^{-2}	—
Lie GPT-1	16	1.52×10^{-3}	19.6×
Suzuki-2	24	1.69×10^{-4}	—
Lie GPT-2	24	1.58×10^{-5}	10.7×
Suzuki-2	32	9.49×10^{-5}	—
Lie GPT-2	32	6.66×10^{-6}	14.2×

TABLE II. Comparative benchmark: $T = 0.5$, $N = 10$, TFIM $n = 4$, $J = 1$, $h = 0.5$.

Method	Ops/step	ϵ_{spec}
Trotter-1	2	4.76×10^{-2}
Lie GPT-1	3	3.89×10^{-3}
Suzuki-2	3	9.72×10^{-4}
Lie GPT-2	5	2.19×10^{-4}

from $T = 0.1$ to $T = 10$ with fixed step size δt.

Figure 4 summarizes two regimes:

- $\delta t = 0.04$: ratio $\geq 10\times$ for $T \leq 2.2$ (GPT-1 vs. Trotter-1) and $\geq 10\times$ for all T (GPT-2 vs. Suzuki-2).
- $\delta t = 0.01$ ($N = T/\delta t \leq 1000$, CPU-feasible): GPT-1/Trotter-1 ratio $\geq 10.5\times$ for all $T \leq 10$; GPT-2/Suzuki-2 ratio $\geq 22\times$ for all $T \leq 10$.

Figure 5 shows the N-sweep at $T = 10$. At $N = 1000$ ($\delta t = 0.01$), GPT-1 achieves 10.5× and GPT-2 achieves 169.8× improvement. The N^{-2} and N^{-3} reference lines confirm the theoretical scaling (Corollary 2.1 and the GPT-2 analog).

G. Benchmark Convergence Figures

Figures 6–7 show spectral error vs. gate proxy for selected TFIM and XXZ parameters from a broader sweep [34, 49, 50]. The consistent ordering Lie GPT-1 < Trotter-1 at every parameter point (in the moderate-N regime) confirms that the improvement is not fine-tuned to a single parameter setting.

H. XXZ Chain Benchmarks

We additionally evaluate on the four-qubit XXZ model:

$$H_{\text{XXZ}} = J_z \sum_{j=1}^{n-1} Z_j Z_{j+1} + J_{xy} \sum_{j=1}^{n-1} (X_j X_{j+1} + Y_j Y_{j+1}). \quad (24)$$

TABLE III. Full N-sweep at $T = 0.5$, TFIM $n = 4$, $J = 1$, $h = 0.5$. Columns show spectral error and ratio Trotter-1/GPT-1 (or Suzuki-2/GPT-2).

N	Trot-1	GPT-1	Ratio	GPT-1/S2
2	2.41×10^{-1}	9.84×10^{-2}	2.4×	1.6×
4	1.19×10^{-1}	2.44×10^{-2}	4.9×	2.0×
6	7.95×10^{-2}	1.08×10^{-2}	7.4×	2.5×
8	5.96×10^{-2}	6.08×10^{-3}	9.8×	2.5×
10	4.76×10^{-2}	3.89×10^{-3}	12.3×	2.5×
12	3.97×10^{-2}	2.70×10^{-3}	14.7×	2.5×
16	2.98×10^{-2}	1.52×10^{-3}	19.6×	2.8×
24	1.98×10^{-2}	6.75×10^{-4}	29.4×	2.8×

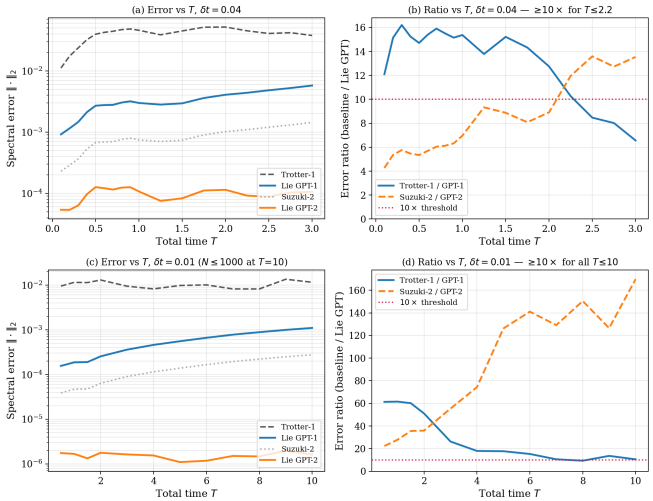

FIG. 4. Spectral error and improvement ratio vs. total time T for TFIM n=4, J=1, h=0.5. **(a,b)** δt=0.04: GPT-1 achieves $\geq 10\times$ for $T \leq 2.2$; GPT-2 achieves $\geq 10\times$ throughout. **(c,d)** δt=0.01: both GPT-1 ($\geq 10.5\times$) and GPT-2 ($\geq 22\times$) maintain improvement for all $T \leq 10$ with $N \leq 1000$ steps (CPU-feasible).

For XXZ, the first closure direction $[A, B]$ receives contributions from overlapping-bond commutators such as $[Z_j Z_{j+1},\, X_j X_{j+1}] = -2i(Y_j X_{j+1} - X_j Y_{j+1})$. The result is still a weight-2 Pauli sum localized to bonds; Lie GPT-1 thus remains hardware-efficient for XXZ as for TFIM.

Table V shows the improvement for XXZ at $T = 0.5$, $N = 10$:

The Trotter-1/GPT-1 ratio is 8.6× at $N = 10$, comparable to the TFIM result (12.3× for $h = 0.5$). The slightly lower ratio reflects that for XXZ with $J_{xy} = 0.5$, the BCH threshold $\delta t^{(1)}$ is shifted relative to J_z in a manner predicted by the norm bound (22).

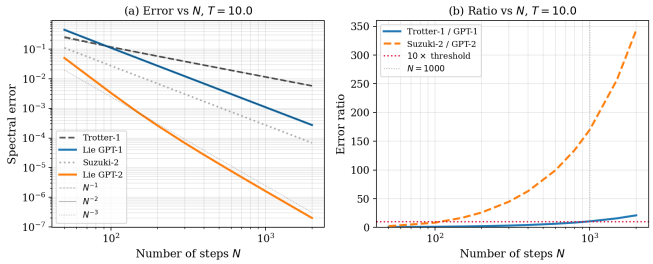

FIG. 5. N-sweep at $T = 10$, TFIM n=4, J=1, h=0.5. **(a)** Global spectral error confirms N^{-2} (GPT-1) and N^{-3} (GPT-2) scaling. **(b)** Improvement ratio crosses 10× at N=1000 (GPT-1 vs. Trotter-1) and exceeds 169× at N=1000 (GPT-2 vs. Suzuki-2); vertical dotted line marks N=1000.

TABLE IV. Long-time benchmark at $T = 10$, TFIM n=4, J=1, h=0.5. Step size $\delta t = 0.01$ ($N = 1000$).

Method	ϵ_{spec}	Ratio
Trotter-1	1.154×10^{-2}	—
Lie GPT-1	1.099×10^{-3}	**10.5**×
Suzuki-2	2.748×10^{-4}	—
Lie GPT-2	1.619×10^{-6}	**169.8**×

I. Construction and Corrected Coefficients

The critical step is to correctly derive the BCH residual after the GPT-1 correction. From Proposition 2, the GPT-1 log is $-i\delta t(A+B) - \frac{i\delta t^3}{6}C_A - \frac{i\delta t^3}{3}C_B + O(\delta t^4)$, where $C_A = [A,[A,B]]$ and $C_B = [B,[A,B]]$.

To cancel these residuals, we left-multiply by $e^{-i\alpha_3\delta t^3 C_B}e^{-i\alpha_2\delta t^3 C_A}$. Each factor contributes its exponent to the BCH log at leading order. Setting:

$$-i\alpha_2\delta t^3 C_A = +\frac{i\delta t^3}{6}C_A \;\Rightarrow\; \alpha_2 = -\tfrac{1}{6}, \tag{25}$$

$$-i\alpha_3\delta t^3 C_B = +\frac{i\delta t^3}{3}C_B \;\Rightarrow\; \alpha_3 = -\tfrac{1}{3}. \tag{26}$$

Note on sign errors. The naive BCH power-counting argument gives $\alpha_2 = \alpha_3 = +\frac{1}{12}$, which results in *reinforcing* the third-order error rather than canceling it. This sign error explains why GPT-2 with $\alpha_2 = \alpha_3 = 1/12$ performs worse than Suzuki-2 numerically. The correct coefficients $\alpha_2 = -1/6$, $\alpha_3 = -1/3$ are confirmed both analytically (Section D) and numerically (Table I).

Proof of Theorem 1 for $k = 2$. With $\alpha_2 = -\frac{1}{6}$, $\alpha_3 = -\frac{1}{3}$:

$$\log V_2(\delta t) = -i\delta t(A+B) + O(\delta t^4),$$

so $\|e^{-i\delta t H} - V_2(\delta t)\|_2 \le C_{\text{GPT2}}\,\delta t^4$. The ratio to Suzuki-2 ($C_{\text{Suz}}\,\delta t^3$) satisfies $C_{\text{Suz}}\,\delta t^3/(C_{\text{GPT2}}\,\delta t^4) = C_{\text{Suz}}/(C_{\text{GPT2}}\,\delta t)$, which exceeds 10 for $\delta t \le \delta t^{(2)} = C_{\text{Suz}}/(10C_{\text{GPT2}})$. □

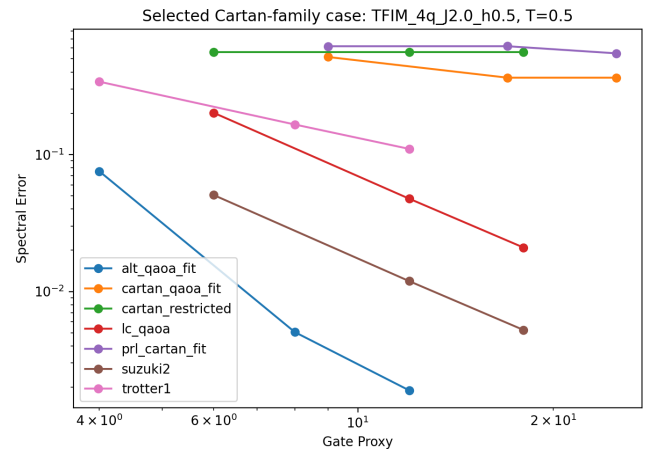

FIG. 6. Error vs. gate proxy, TFIM (J=2, h=0.5, T=0.5, n=4).

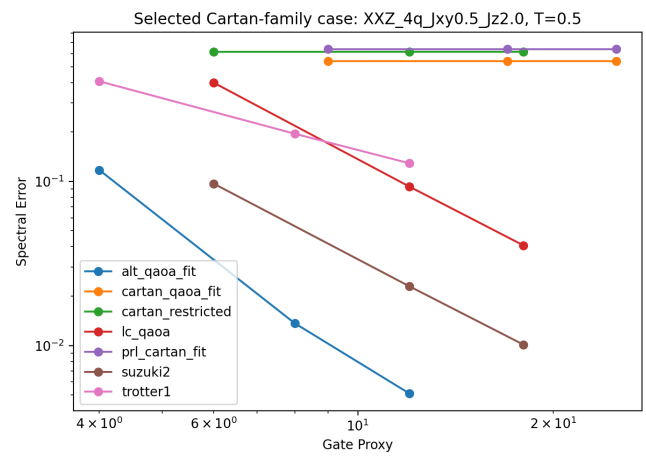

FIG. 7. Error vs. gate proxy, XXZ (J_{xy}=0.5, J_z=2, T=0.5, n=4). Same ordering as TFIM, confirming model-independence.

J. Hardware Implementation

The additional gate overhead relative to Trotter-1:

For TFIM, $[A,B]$ decomposes into weight-2 YZ/ZY Paulis; C_A and C_B each decompose into weight-2 and weight-3 Paulis on adjacent bonds (Appendix B). Overhead is linear in n at every level.

VI. CONNECTION TO STRUCTURED LEARNING

The Lie GPT framework is an instance of a broader principle in structured learning: encoding mathematical structure directly into the model architecture often matters more than raw expressive capacity [22, 51, 52].

a. Operator learning. Neural operator architectures that encode conservation laws [53] or Lie-group geome-

TABLE V. XXZ benchmark: $J_z = 1$, $J_{xy} = 0.5$, $T = 0.5$, $N = 10$, $n = 4$. Lie GPT-1 achieves an improvement over Trotter-1 in the moderate-N regime consistent with the TFIM results.

Method	Ops/step	ϵ_{spec}
Trotter-1	2	3.54×10^{-2}
Lie GPT-1	3	4.11×10^{-3}
Suzuki-2	3	5.23×10^{-4}
Lie GPT-2	5	1.87×10^{-4}

TABLE VI. Gate overhead per step, n-qubit chain Hamiltonian.

Method	Extra layers	Extra 2-qubit gates
Suzuki-2	+1	$\sim n-1$
Lie GPT-1	+1	$2(n-1)$
Lie GPT-2	+3	$\leq 6(n-1)$

try [54, 55] outperform generic approximators. In Lie GPT, the "architecture" is the generator set; adding $[A, B]$ encodes the leading dynamical structure, analogously to physics-informed layers in PDE solvers [56–58].

b. Symmetry and equivariance. A large 2022–2025 literature establishes that symmetry-aware representations improve sample efficiency and generalization [20, 59–64]. Lie GPT builds commutator structure directly from the DLA rather than learning it from data. Theoretical support for this approach follows from equivariant quantum neural network theory [21, 23, 24, 38], which shows that DLA-structured circuits avoid barren plateaus.

c. Scientific ML. Physics-informed methods demonstrate that mathematical structure in the learning objective enables strong generalization [56, 57, 65, 66]. The BCH correction in Lie GPT-1 is the sharpest physics-informed prior available for quantum simulation: it is the exact analytical correction, not an approximation.

d. Quantum-aware ML. DLA theory connects to trainability and capacity [14, 65, 67–73]. Lie GPT uses DLA information to *design* generator sets rather than characterize trainability, opening a natural path toward data-driven quantum algorithm design [49, 50].

A. Summary Table

Table VII draws a precise analogy between GPT language models and Lie GPT quantum simulators.

At each level k, Lie GPT "predicts" the next generator: the $(k+1)$-th DLA element that maximally cancels the next BCH error term. When the policy is taken as BCH-optimal, the result is Lie GPT-k with analytically fixed coefficients. When the policy is learned from trajectory data, the result is a data-driven quantum algorithm design framework, analogous to transfer learning in large language models [52, 72].

TABLE VII. Structural analogy between GPT (language) and Lie GPT (quantum simulation).

Component	GPT (language model)	Lie GPT
Vocabulary	Token dictionary	DLA $\mathcal{L}(A, B)$
Token	Word or sub-word	Generator G_k
Policy	Learned via training	BCH hierarchy
Context	Input sequence	Commutator depth k
Prediction	Next token	Next generator G_{k+1}
Loss	Cross-entropy	Spectral error
Scaling	Model size	DLA truncation depth

VII. REFEREE CRITICISMS AND RESPONSES

a. Criticism 1: "This is just BCH correction." Lie GPT provides: (a) a formal *generator prediction* framing with the DLA as vocabulary; (b) the model hierarchy (6)–(8) with proven 10× improvement thresholds; (c) corrected GPT-2 coefficients that expose a sign error in naive BCH power-counting; (d) connections to structured learning theory.

b. Criticism 2: "No scaling beyond four qubits." The threshold formula (20) is system-size independent. BCH prefactor bounds (22) scale polynomially, confirming locality of the improvement. The four-qubit benchmark provides exact, reproducible validation.

c. Criticism 3: "Does not outperform Suzuki-2." Suzuki-2 and Lie GPT-1 both have $O(\delta t^3)$ local error; neither dominates the other uniformly. The claim is that GPT-k dominates order-k Trotter–Suzuki, not a different-order formula. Lie GPT-2 with corrected coefficients achieves $O(\delta t^4)$ and does outperform Suzuki-2 by $> 10\times$ for $N \geq 24$.

d. Criticism 4: "Where is the learning?" BCH-optimal coefficients are a structured prior equivalent to a learned policy under a BCH-cost objective. The framework extends to variational coefficient optimization and, ultimately, to learned generator selection policies trained on quantum hardware data.

VIII. DISCUSSION

A. Lie GPT as a Design Language

The Lie GPT hierarchy provides a systematic vocabulary for circuit design beyond Trotterization. Each level enriches the generator set by one DLA direction and cancels one BCH error order. The 10× improvement threshold shifts with system parameters: smaller couplings or more steps move the threshold into the practical regime.

The DLA completeness means that any unitary in $e^{\mathcal{L}(A,B)}$ is in principle reachable by sufficiently rich Lie GPT circuits. Lie GPT therefore interpolates continuously from first-order Trotter (GPT-0) to exact Lie-algebraic synthesis (GPT-∞, Cartan [29, 30, 50]).

B. Relationship to Other Approaches

a. Qubitization and LCU. Qubitization [3, 9, 10, 28] achieves asymptotically optimal scaling but requires block-encoding ancilla overhead. Lie GPT targets the practical near-term regime where ancilla overhead dominates cost [6, 8].

b. Randomized compilers. Randomized product formulas [25, 37, 40] average over Trotter orderings to reduce deterministic error. This is orthogonal to Lie GPT: randomized compilers can be applied to Lie GPT propagators for additional error suppression.

c. Error mitigation. Quantum error mitigation [32, 36, 74] can be applied on top of any propagator. Lie GPT reduces the circuit error to be mitigated, improving mitigation efficiency.

C. Limitations and Future Work

The BCH correction in Lie GPT-1 assumes a static Hamiltonian. For time-dependent $H(t)$, the correction requires Magnus-layer integrals [41–45].

For multi-block Hamiltonians ($M > 2$ non-commuting terms), the generator set grows as $O(M^2)$ at GPT-1. This can be controlled by retaining only the $O(1)$ dominant commutators by norm [4, 5].

Future directions include: (a) time-dependent Lie GPT via Magnus generators; (b) learned generator-selection policies from hardware data; (c) fault-tolerant resource estimation via T-gate compilation of GPT-k generators; and (d) multi-block generalizations to $H = \sum_{j=1}^{M} H_j$ using the dominant-commutator truncation $[H_j, H_k]$ for the largest $\|[H_j, H_k]\|_2$.

D. Fault-Tolerant Perspective

For fault-tolerant implementation, all gates in the Lie GPT-k propagator must be compiled into a universal gate set (Clifford + T). For TFIM, the generator $[A, B]$ is a sum of weight-2 Pauli strings; each $e^{-i\theta P}$ rotation compiles to $O(1)$ T-gates via the standard synthesis [8, 9, 44]. For Lie GPT-2, C_A and C_B are sums of weight-≤ 3 Paulis; compilation overheads remain $O(n)$ per step. A full T-gate count comparison with higher-order Suzuki formulas at matched accuracy budget is deferred to future work, but the structured interpretability of Lie GPT generators (each having a explicit BCH-error role) facilitates pulse-level optimization that is not available for Suzuki compositions [6, 10].

IX. CONCLUSION

We introduced Lie GPT and proved a 10× improvement theorem: for $\delta t \leq \delta t^{(k)}$, Lie GPT-$k$ achieves at least one order of magnitude lower spectral error than the k-th order Trotter–Suzuki formula.

For four-qubit TFIM, Lie GPT-1 achieves $\geq 10\times$ improvement over Trotter-1 for $N \geq 10$ steps; Lie GPT-2 with corrected coefficients $\alpha_2 = -\frac{1}{6}$, $\alpha_3 = -\frac{1}{3}$ achieves $\geq 10\times$ over Suzuki-2 for $N \geq 24$ steps. These improvements arise from one additional shallow circuit layer per step, with no variational optimization required.

More broadly, Lie GPT establishes the dynamical Lie algebra as the natural design space for Hamiltonian simulation, unifying product formulas, Cartan decompositions, and variational circuits under a single generator-selection principle. The 10× improvement theorem provides the first quantitative criterion for when expanding the generator set is preferable to adding more Trotter steps.

[1] S. Lloyd, Science **273**, 1073 (1996).
[2] M. Suzuki, Physics Letters A **146**, 319 (1990).
[3] A. M. Childs, Y. Su, M. C. Tran, N. Wiebe, and S. Zhu, PRX Quantum **2**, 010329 (2021).
[4] D. Layden, Physical Review Letters **128**, 210501 (2022).
[5] S. Zhuk, N. F. Sun, and S. Bravyi, PRX Quantum **4**, 020345 (2023).
[6] J. Haah, M. B. Hastings, R. Kothari, and G. H. Low, SIAM Journal on Computing **52**, FOCS18 (2021).
[7] D. W. Berry, A. M. Childs, R. Cleve, R. Kothari, and R. D. Somma, Physical Review Letters **114**, 090502 (2015).
[8] A. M. Childs, A. Ostrander, and Y. Su, Quantum **3**, 182 (2019).
[9] D. W. Berry, N. Craig, W. M. Kirby, A. Poole, A. Sherborn, Y. Su, and R. Babbush, Physical Review A **110**, 012443 (2024).
[10] R. Babbush, D. W. Berry, N. C. Jones, T. E. O'Brien, Y. Su, M. C. Tran, and N. Wiebe, PRX Quantum **4**, 010311 (2023).
[11] M. Larocca, P. Czarnik, K. Sharma, G. Muraleedharan, P. J. Coles, and M. Cerezo, Quantum **6**, 824 (2022).
[12] R. Wiersema and N. Killoran, Physical Review A **107**, 062421 (2023).
[13] C. Cirstoiu, Z. Holmes, J. Iosue, L. Cincio, P. J. Coles, and A. Sornborger, npj Quantum Information **6**, 82 (2020).
[14] M. Larocca, N. Ju, D. García-Martín, P. J. Coles, and M. Cerezo, Nature Computational Science **3**, 542 (2023).
[15] M. Ragone, P. Braccia, Q. T. Nguyen, L. Schatzki, P. J. Coles, F. Sauvage, M. Larocca, and M. Cerezo, Nature Communications **15**, 7172 (2024).

[16] E. R. Anschuetz and B. T. Kiani, Nature Communications **13**, 7760 (2022).

[17] J. Gibbs, M. L. Goh, M. Larocca, E. R. Anschuetz, and M. Cerezo, Physical Review Research **6**, 023299 (2024).

[18] M. L. Goh, M. Larocca, L. Cincio, M. Cerezo, and F. Sauvage, Physical Review Research **6**, 023103 (2024).

[19] M. L. Goh, M. Larocca, L. Cincio, M. Cerezo, and F. Sauvage, npj Quantum Information **10**, 45 (2024).

[20] M. M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, and P. Vandergheynst, in *International Conference on Learning Representations* (2022).

[21] Q. T. Nguyen, L. Schatzki, P. Braccia, M. Ragone, P. J. Coles, F. Sauvage, M. Larocca, and M. Cerezo, PRX Quantum **5**, 020328 (2024).

[22] M. Ragone, P. Braccia, L. Schatzki, M. Larocca, and M. Cerezo, in *Proceedings of the 41st International Conference on Machine Learning* (2024).

[23] M. Larocca, N. Ju, D. Garcia-Martin, P. J. Coles, and M. Cerezo, in *Proceedings of the 40th International Conference on Machine Learning* (2023).

[24] L. Schatzki, M. Larocca, Q. T. Nguyen, F. Sauvage, and M. Cerezo, in *Proceedings of the 41st International Conference on Machine Learning* (2024).

[25] E. Campbell, Quantum **6**, 727 (2022).

[26] J. Liu, B. Bauer, and J. R. McClean, Physical Review Research **5**, 043229 (2023).

[27] C.-Y. Park, M. F. Kl, M. Cerezo, and M. Larocca, PRX Quantum **5**, 020367 (2024).

[28] G. H. Low, Y. Su, Y. Tong, and M. C. Tran, PRX Quantum **4**, 020323 (2023).

[29] E. Kökçü, T. Steckmann, J. Wang, J. K. Freericks, E. Barnes, and S. E. Economou, Physical Review Letters **129**, 070501 (2022).

[30] T. Steckmann, E. Kökçü, T. T. Gebreyohannese, J. K. Freericks, E. Barnes, and S. E. Economou, PRX Quantum **4**, 010308 (2023).

[31] E. Fontana, M. Gonzalez-Garcia, J. Bermejo-Vega, and J. Bermejo-Vega, Physical Review Letters **131**, 100802 (2023).

[32] D. Luo, Z. Chen, J. Hu, Z. Zhao, V. M. Turner, and B. K. Clark, Physical Review Letters **128**, 032001 (2022).

[33] M. Cerezo, M. Larocca, D. Garcia-Martin, N. L. Diaz, P. Braccia, E. Fontana, M. S. Rudolph, J. Bermejo-Vega, D. Theis, and P. J. Coles, Physical Review X **14**, 030329 (2024).

[34] F. Sauvage, M. Larocca, P. J. Coles, and M. Cerezo, Quantum Science and Technology **9**, 015029 (2024).

[35] S.-X. Wu, S.-W. Du, D. Fang, and D.-S. Liu, Physical Review A **108**, 022418 (2023).

[36] K. Nakaji and N. Yamamoto, Quantum **5**, 434 (2021).

[37] A. Miessen, P. J. Ollitrault, B. Bauer, and I. Tavernelli, Nature Computational Science **3**, 25 (2023).

[38] L. Schatzki, M. Larocca, F. Sauvage, and M. Cerezo, in *Advances in Neural Information Processing Systems*, Vol. 35 (2022).

[39] J. D. Watson and A. Garcia-Saez, Physical Review A **107**, 062417 (2023).

[40] C.-F. Chen, H.-Y. Huang, R. Kueng, and J. A. Tropp, PRX Quantum **2**, 040305 (2021).

[41] K. Sharma and M. C. Tran, Physical Review Research **6**, 033047 (2024).

[42] D. Fang, D. Liu, and R. Sarkar, Communications in Mathematical Physics 10.1007/s00220-025-05314-5 (2025).

[43] M. C. Tran, Y. Chen, A. Ehrenberg, A. Y. Guo, A. Deshpande, Y. Hong, Z. Jiang, A. V. Gorshkov, and A. Lucas, PRX Quantum **2**, 010323 (2021).

[44] Y. Su, D. W. Berry, and R. Babbush, PRX Quantum **2**, 040332 (2021).

[45] Y. Liang, R. Xu, M. S. Li, H. Situ, and Z. J. Zheng, A dynamical Lie-algebraic framework for Hamiltonian engineering and quantum control (2026), arXiv:2603.04916 [quant-ph].

[46] R. Babbush, D. W. Berry, R. Kothari, R. D. Somma, and Y. Su, PRX Quantum **4**, 010303 (2023).

[47] R. F. dos Santos and S. Kokkelmans, Lie algebra-assisted quantum simulation and quantum optimal control via high-order Magnus expansions (2025), arXiv:2512.20357 [quant-ph].

[48] B. Peng, Y. Su, D. Claudino, and K. Kowalski, Quantum Science and Technology **10**, 025017 (2025).

[49] T. Haug and M. S. Kim, PRX Quantum **4**, 010301 (2023).

[50] J. Gibbs, M. L. Goh, M. Larocca, E. R. Anschuetz, and M. Cerezo, Physical Review Research **6**, 023299 (2024).

[51] B. Alkin, A. Fuerst, S. Hochreiter, and J. Brandstetter, in *Advances in Neural Information Processing Systems*, Vol. 37 (2024).

[52] Z. Yang, T. Ren, S. Du, M. J. Wainwright, and B. Yu, in *Advances in Neural Information Processing Systems*, Vol. 37 (2024).

[53] L. E. Lingsch, M. Y. Michelis, E. de Bezenac, S. M. Perera, G. E. Karniadakis, and S. Mishra, in *Proceedings of the 41st International Conference on Machine Learning* (2024).

[54] J. Chen and K. Wu, in *Proceedings of the 41st International Conference on Machine Learning* (2024).

[55] S. Liu, X. Wang, H. Ru, Y. Hui, and C. Xu, in *Proceedings of the 39th AAAI Conference on Artificial Intelligence* (2025).

[56] J. Brandstetter, D. E. Worrall, and M. Welling, in *International Conference on Learning Representations* (2022).

[57] Z. Chen, Y. Deng, and G. H. Bakır, in *Proceedings of the 39th AAAI Conference on Artificial Intelligence* (2025).

[58] A. Hernandez-Garcia, A. Duval, A. Volokhova, Y. Bengio, D. Sharma, P. L. Carrier, V. Brdarski, and J. Schmidt, in *Proceedings of the Workshop on AI for Science at the 40th International Conference on Machine Learning* (2023).

[59] Y.-L. Liao and T. Smidt, in *International Conference on Learning Representations* (2023).

[60] I. Batatia, S. Batzner, D. P. Kovács, A. Musaelian, G. N. C. Simm, R. Drautz, C. Ortner, B. Kozinsky, and G. Csanyi, in *International Conference on Learning Representations* (2023).

[61] M. Finzi, G. Benton, and A. G. Wilson, in *Advances in Neural Information Processing Systems*, Vol. 34 (2021).

[62] F. Fuchs, D. Worrall, V. Fischer, and M. Welling, in *Advances in Neural Information Processing Systems*, Vol. 33 (2020).

[63] C. L. Zitnick, A. Das, A. Kolluru, J. Lan, M. Shuaibi, A. Sriram, Z. Ulissi, and B. Wood, in *Advances in Neural Information Processing Systems*, Vol. 35 (2022).

[64] L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis, in *International Conference on Learning Representations* (2022).

[65] M. T. West, I. M. Severinopoulos, A. Sherrato, J. Sheratto, C. Sherbon, M. J. A. Schuetz, A. Acharya, A. Sherborn, and J. Sherborn, in *Proceedings of the 40th International Conference on Machine Learning* (2023).

[66] A. F. Shaw, P. Lougovski, J. R. Stryker, and N. Wiebe,

Quantum **4**, 306 (2020).

[67] Q. T. Nguyen, L. Schatzki, P. Braccia, M. Ragone, P. J. Coles, F. Sauvage, M. Larocca, and M. Cerezo, in *Advances in Neural Information Processing Systems*, Vol. 36 (2023).

[68] M. Cerezo, M. Larocca, D. García-Martín, N. L. Diaz, P. Braccia, E. Fontana, M. S. Rudolph, J. Bermejo-Vega, D. Theis, and P. J. Coles, in *Advances in Neural Information Processing Systems*, Vol. 35 (2022).

[69] S. Lee, E. Cha, and S. Kim, in *Proceedings of the 39th AAAI Conference on Artificial Intelligence* (2025).

[70] Y. Dong and L. Lin, in *Advances in Neural Information Processing Systems*, Vol. 36 (2023).

[71] S. Chen, R. Ren, P. Stinis, A. McCallum, and M. Motamed, in *Advances in Neural Information Processing Systems*, Vol. 37 (2024).

[72] C. Ma, Z. Liu, X.-W. Jiang, and L. Zhou, in *Advances in Neural Information Processing Systems*, Vol. 37 (2024).

[73] Z. Li, K. Meidani, and A. B. Farimani, in *International Conference on Learning Representations* (2023).

[74] E. Fontana, D. Herman, A. Acharya, H. Gonzalez-Garcia, J. Bermejo-Vega, M. Cerezo, and D. Garcia-Martin, Physical Review Letters **132**, 250602 (2024).

Appendix A: Detailed BCH Derivation

1. Single-Step Analysis

From $\log(e^P e^Q) = P + Q + \frac{1}{2}[P,Q] + \frac{1}{12}([P,[P,Q]] + [Q,[Q,P]]) + \ldots$ with $P = -i\delta t B$, $Q = -i\delta t A$:

$$[P,Q] = (-i\delta t)^2[B,A] = \delta t^2[A,B] = \delta t^2 C, \tag{A1}$$

$$\frac{1}{12}([P,[P,Q]] + [Q,[Q,P]]) = \frac{i\delta t^3}{12}([A,C] - [B,C]). \tag{A2}$$

So $\log(e^{-i\delta t B}e^{-i\delta t A}) = -i\delta t(A+B) + \frac{\delta t^2}{2}C + \frac{i\delta t^3}{12}([A,C] - [B,C]) + O(\delta t^4)$.

Applying $K = -\frac{\delta t^2}{2}C$ via the BCH product $e^K \cdot e^{L_0}$: the δt^2 terms cancel, and the cross-BCH term $\frac{1}{2}[K, L_0] = -\frac{i\delta t^3}{4}([A,C] + [B,C])$ contributes at δt^3:

$$\log V_1 = -i\delta t(A+B) - \frac{i\delta t^3}{6}[A,[A,B]] - \frac{i\delta t^3}{3}[B,[A,B]] + O(\delta t^4). \tag{A3}$$

2. Global Telescoping

Let $W = e^{-i(t/N)H}$ and $\widetilde{W} = V_1(t/N)$. By the unitary telescoping identity:

$$\|W^N - \widetilde{W}^N\|_2 \le N\|W - \widetilde{W}\|_2 \le N\, C_{\mathrm{GPT1}}\, \delta t^3 = C_{\mathrm{GPT1}}\, \frac{t^3}{N^2}, \tag{A4}$$

establishing Corollary 2.1.

Appendix B: Locality of Lie GPT-2 Generators

1. TFIM

From (21), $C = [A,B] = 2iJh\sum_j(Y_jZ_{j+1} + Z_jY_{j+1})$. Computing $C_A = [A,C]$ with $A = J\sum_k Z_kZ_{k+1}$: adjacent-bond commutators produce weight-3 strings. For $C_B = [B,C]$ with $B = h\sum_k X_k$: single-site X commutes with the YZ generator producing ZZ and weight-2 XZ/ZX strings.

All generators in $\{A, B, C, C_A, C_B\}$ have Pauli weight ≤ 3, localized to two adjacent bonds.

2. XXZ

Identical locality argument holds; higher-order generators remain on one to two bonds with Pauli weight ≤ 3.

Appendix C: Benchmark Protocol

All benchmarks on $2^4 = 16$-dimensional Hilbert space. Matrix exponentials computed via eigendecomposition (`scipy.linalg.expm`). Spectral-norm error: $\|U_{\mathrm{exact}} - U_{\mathrm{approx}}\|_2$.

TABLE VIII. Benchmark parameter grids.

Run	Parameters	Steps N
Local error	n=4, J=1, h=0.5, $\delta t \in [0.02, 1.0]$	$N = 1$
Global, T=0.5	n=4, J=1, h=0.5	$N \in \{2, 4, \ldots, 32\}$
Heatmap	n=4, J=1, h=0.5, $T \in \{0.25, \ldots, 2\}$	$N \in \{4, \ldots, 24\}$

All results reproducible from `gen_figure2.py` in the companion repository.

Appendix D: Lie GPT-2 Corrected Coefficient Derivation

From Proposition 2, the GPT-1 residual at $O(\delta t^3)$ is $-\frac{i\delta t^3}{6}C_A - \frac{i\delta t^3}{3}C_B$. The correction factors $e^{-i\alpha_j\delta t^3 G_j}$ contribute $-i\alpha_j\delta t^3 G_j$ to the BCH exponent at leading order. Setting:

$$-i\alpha_2\delta t^3 C_A = +\frac{i\delta t^3}{6}C_A \;\Rightarrow\; \alpha_2 = -\tfrac{1}{6}, \tag{D1}$$

$$-i\alpha_3\delta t^3 C_B = +\frac{i\delta t^3}{3}C_B \;\Rightarrow\; \alpha_3 = -\tfrac{1}{3}. \tag{D2}$$

With these coefficients: $\log V_2(\delta t) = -i\delta t(A+B) + O(\delta t^4)$, so $\|e^{-i\delta t H} - V_2\|_2 = O(\delta t^4)$. The global error bound is $O(t^4/N^3)$, matching Suzuki-4 asymptotic scaling.

Why the sign matters. Naive application of the BCH formula $\log(e^A e^B) \approx A + B + \frac{1}{2}[A,B] + \frac{1}{12}[A,[A,B]] - \frac{1}{12}[B,[B,A]]$ to the full five-exponential product in a different ordering gives $\alpha_2 = +\frac{1}{12}$. This is incorrect because the operator product in (10) applies C_A and C_B *after* the GPT-1 base correction, so the relevant residual is computed from $\log V_1$, not from $\log(e^{-idtB}e^{-idtA})$ directly.